

Project - 3 Understanding LRU-IPV - Group 19

Group Members

1. Krutyanjay Shinde
- ASU ID :- 1234108189
2. Vaishnavi Sardeshpande
- ASU ID :- 1233932520
3. Veena Madhuri Tumuluri
- ASU ID :- 1232463962

1. Answer the following questions based on the Least Recently Used (LRU) Replacement Policy as discussed in class and from the reading of the paper “Insertion and Promotion for Tree-Based PseudoLRU Last-Level Caches”.

a. What is the LRU replacement policy? Why do we need a replacement policy for a set-associative cache and a fully associative cache?

The Least Recently Used (LRU) replacement policy is a way used in caches to decide which data block to remove or when new data needs to be loaded. It works on the principle that the block that has not been used for the longest time (which is quite literally, least recently used) is the least likely to be needed again soon, so it should be replaced. In this policy, every time a cache block is accessed, it is marked as the most recently used (MRU), and when the cache becomes full, the block that was used the longest time ago (the LRU block) is evicted.

We need a replacement policy for a set-associative and fully associative cache because more than one block can fit into the same cache set. When all the spaces in that set are full and a new block needs to be stored, the cache must decide which old block to remove. The replacement policy provides a rule for making this decision. For example, in the LRU policy, the block that was used the longest time ago is replaced to make room for the new one.

b. As discussed in the paper, why does the LRU replacement policy need an improvement? Explain the concept of insertion/promotion vector and how it improves the LRU replacement policy.

The insertion/promotion vector (IPV) is a way to improve how LRU decides where to place new blocks and how to move re-referenced ones within the cache. In the usual LRU way, every new block is inserted at the most recently used (MRU) position, and every time it is accessed again, it is promoted back to MRU. However, this isn't always the best choice is what the author and previous works have proposed. This is because some blocks may be used only once or a few times. The IPV provides more flexibility by allowing different positions for inserting and promoting blocks. It is a vector that defines where a block should be moved when accessed and where a new block should be inserted in the cache. For example, a new block could be inserted at the LRU position that is at the bottom of the stack, and if it is accessed again, it could be promoted to the middle, and

then to the MRU position after repeated use. This gradual movement helps the cache keep frequently used blocks longer while quickly evicting blocks that are rarely reused.

c. Based on your understanding of the paper, explain the key ideas behind the tree-based PseudoLRU with the help of the algorithms provided in Figures 5 and 6 and Section 3.1.

The key idea behind the tree-based PseudoLRU (PLRU) replacement policy is to provide a simpler and more hardware-efficient way to approximate the behaviour of the LRU policy. It uses a binary tree where each leaf node represents a cache block in a set and each internal node contains a single plru bit. These bits help identify which block was used least recently and which one should be replaced next.

Figure 5:

```

find_plru ( r ) r is the root node of the binary tree
    p = r
    while ( p is not a leaf )
        if ( p's plru bit == 1 )
            p = right child of p
        else
            p = left child of p
        end if
    end while
    return the block associated with p

```

In the above algorithm, when the cache needs to choose a block for replacement, the algorithm starts at the root node and checks the plru bit at each internal node. If the bit is 1, it moves to the right child, and if the bit is 0, it moves to the left child. This process continues until it reaches a leaf node, which represents the cache block to be replaced and it is the PseudoLRU block.

Figure 6:

```

promote ( p ) p is the leaf node associated with a block to be promoted
    while ( p is not the root )
        if ( p is a left child )
            p's plru bit = 1
        else
            p's plru bit = 0
        end if
        p = parent of p
    end while

```

In the above algorithm, when a block is accessed, the algorithm updates the plru bits along the path from that block to the root so that they point away from the recently used block. If the block is a left child, its parent's bit is set to 1, and if it is a right child, its parent's bit is set to 0. This ensures that the accessed block is not selected for replacement soon and becomes the pseudo most recently used (PMRU) block.

d. Based on Figure 7 and 8, and Section 3.2, which describes the `find_index(p)` algorithm to compute the position of a block `p` which is the leaf node, in the PseudoLRU recency stack. Explain how the algorithm uses the PLRU bits and the structure of the binary tree to determine the recency position of `p`.

Figure 7:

```

find_index ( p ) p is the leaf node associated with a block
  x = 0
  i = 0
  q = p
  while ( q is not the root )
    if ( q is a right child )
      if ( q's parent's plru bit is 1 ) set bit i of x to 1
    else
      if ( q's parent's plru bit is 0 ) set bit i of x to 1
    end if
    q = the parent of q
    i = i + 1
  end while
  return x

```

The `find_index(p)` algorithm above, uses the plru bits stored in the internal nodes of the binary tree and the structure of that tree to figure out the recent position of a cache block, represented by leaf node `p`, in the PseudoLRU (PLRU) recency stack.

The algorithm starts from the leaf node and moves upward toward the root. At each level, it checks if the node is a right child, and then the value of its parent's plru bit. Depending on these two values, it sets specific bits in a binary counter `x`. This counter is updated as the algorithm moves up the tree and finally represents the block's position in the recency order. So the pattern of 0s and 1s formed by these bits encodes how recently that block was used relative to others.

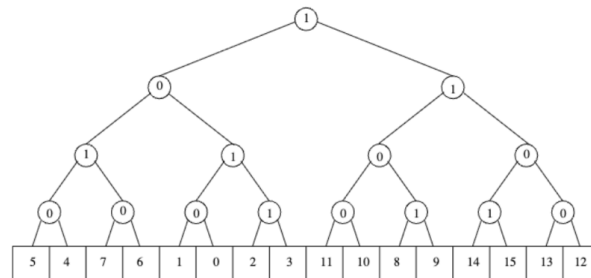


Figure 8: Example PseudoLRU Tree. Blocks are labeled with their positions in the PseudoLRU recency stack derived from internal node plru bit values.

In Figure 8, the tree shows how the plru bits of internal nodes decide the relative recency of each block. For example, the root and intermediate node bits (0 or 1) create a unique binary path for each leaf. Blocks with smaller numbers, like 0 or 1, are more recently used, while those with larger numbers, like 14 or 15, are least recently used.

Thus, by drawing from both the plru bit values and the binary tree structure, the algorithm determines the recency position of each block.

e. Based on Sections 2.4 and 2.5, answer this question. Figure 3 shows the transition graph for an insertion/promotion vector - [0 0 1 0 3 0 1 2 1 0 5 1 0 0 1 11 13]. Explain how the transition graph in Figure 3 differs from the LRU transition graph shown in Figure 2.

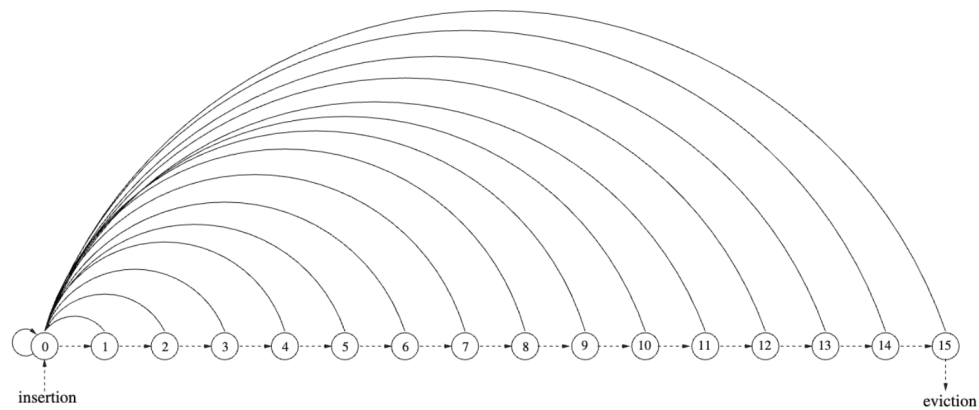


Figure 2: Transition Graph for LRU. A solid edge points to the new position for an accessed or inserted block. A dashed edge shows where a block is shifted when another block is moved to its position.

The LRU graph in the above Figure (Figure 2) shows a pattern where every time a block is used, it always moves straight to the most recently used, that is the MRU position at the top represented by insertion, and all other blocks move down one place. This makes the movement regular and predictable. This is the foundational logic.

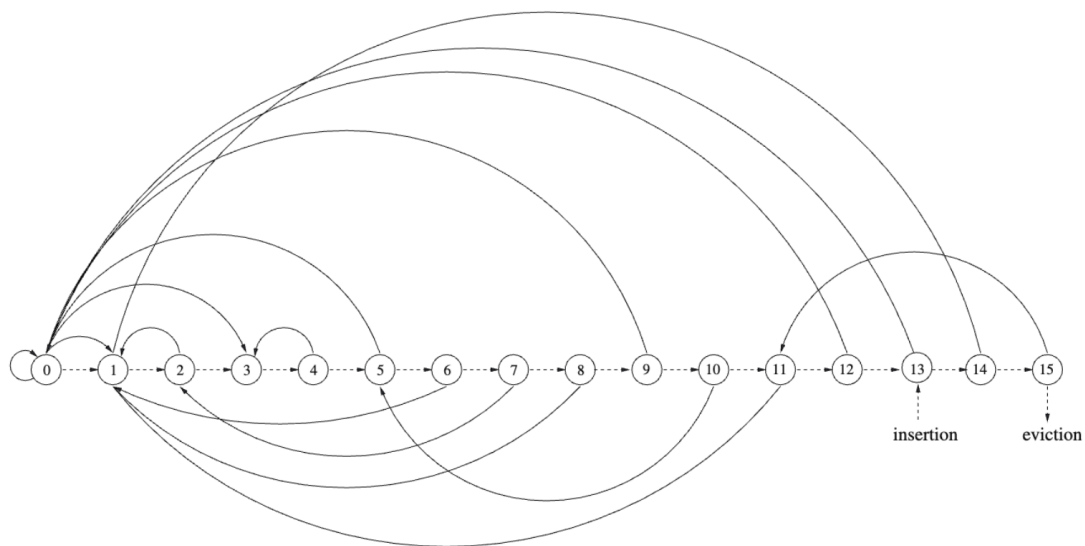


Figure 3: Transition Graph for Vector [0 0 1 0 3 0 1 2 1 0 5 1 0 0 1 11 13]

In the above Figure 3, the movement of blocks is based on the insertion/promotion vector (IPV) [0 0 1 0 3 0 1 2 1 0 5 1 0 0 1 11 13]. Each number in the vector tells where a block should move when it is accessed from a certain position. For example, a block at position 2 moves to position 1, and a new block is inserted at position 13 instead of directly at the top. This makes the pattern more flexible unlike predictable in LRU that is depicted in Figure 2.

Also the vector has 17 elements while the graph has 16 positions because the first 16 numbers describe how existing blocks move when they are used, and the last number represents where a new block is inserted in the cache which is 13 here.

2. Analyze the code and logic for the implementation of the functions (reset(), touch(), invalidate(), getVictim()) and replacement policies from the files as mentioned. Based on your understanding, answer the following questions.

a. Based on your understanding after reading base_set_assoc.cc and base_set_assoc.hh, explain the specific roles of invalidate() and findVictim() functions in terms of cache management.

Here is my understanding of the invalidate() and findVictim() as used in the files provided.

The invalidate() function is called when a cache block becomes invalid which could be the case if the data is no longer valid due to an update from memory. This function first marks the block as invalid and reduces the count of valid cache tags. Then, it tells the replacement policy that this block can now be reused by calling the invalidate() function. So, it clears the old block's status so the cache knows it can replace it with new data when needed.

The findVictim() function is used when there is a cache miss and a new block is expected to be loaded into the cache. It works by asking the indexing policy to list all possible blocks in the set where the new block could be loaded. Then, it calls the getVictim() function, which chooses the best block to evict based on the replacement policy. So the findVictim() finds the block that should be replaced to make way for other new data.

b. The lru_rp.cc and lru_rp.hh files implement LRU (Least Recently Used) cache replacement policy. Based on your understanding of the implementation, explain the specific roles of each of the following functions in implementing the LRU policy: reset(), touch(), invalidate(), and getVictim().

reset(): Called when a block is inserted. It sets that block's lastTouchTick to curTick() which tells the program that this block was just used now. So it starts as most-recently used.

touch(): Called on a cache hit. It updates lastTouchTick to curTick(), moving the block to most recently used (MRU) position in LRU order.

invalidate(): is called when a block is invalidated. This resets lastTouchTick, marking it as the oldest or the least recently used so it will be the one to evict next.

getVictim(): calls when there is a miss to choose a block to evict. It scans all candidate entries and selects the one with the smallest lastTouchTick which is nothing but the one used the longest time ago.

3. Answer the following question based on your understanding of the paper “Insertion and Promotion for Tree-Based PseudoLRU Last-Level Caches” and your answers to questions 1 and 2 above.

Question: Sketch out your plans to implement the LRU-IPV (Least Recently Used-Insertion/Promotion Vector). Describe the functions you will implement and the data structures/variables you will use. Give as much implementation detail as possible when describing your functions, as your answer will be graded on the correctness of your description.

Our goal is to preserve the simplicity of LRU while adding flexibility to control where a new block is inserted and how far a recently used block is promoted. This is what we learnt from the paper.

To implement the Least Recently Used – Insertion/Promotion Vector (LRU-IPV) policy, we would first need to understand what LRU does. And what we want LRU-IPV to do.

1. Overview of the LRU-IPV Concept

Traditional LRU always inserts new blocks and promotes hits to the Most Recently Used (MRU) position. In contrast, LRU-IPV uses a vector $V[0..k]$, where k is the associativity of the set.

- Each entry $V[i]$ (for $i < k$) defines the new position for a block that was previously at position i when it is accessed which is called a promotion.
- $V[k]$ defines the insertion position for a newly fetched block.

This approach allows non-MRU insertions and partial promotions, as described in Section 2.3 of the paper. It helps reduce cache pollution caused by zero reuse or short lived blocks while retaining frequently reused ones closer to MRU.

The implementation steps mainly involve designing efficient data structures to represent recency positions, updating those structures through the four key cache management functions (`reset()`, `touch()`, `invalidate()`, and `getVictim()`), and ensuring the metadata remains consistent across all blocks in a cache set.

2. Data Structures and Variables

To represent the recency ordering, each cache set should maintain metadata (ideally lightweight) shared among all its blocks, similar to the `tree_plru` structure mentioned in the project guide. The key elements are:

- Associativity parameter (k) – number of ways in the set.
- Insertion/Promotion Vector ($V[0..k]$) –: An array of $k+1$ integers that defines the insertion and promotion behavior. This is loaded during initialization.
- Recency position mapping: To avoid physically moving data, we will use 2 two integers array.
 - `pos_of_way[k]`: An array where the index is the way and the value is its current recency position ($0 = \text{MRU}$, $k-1 = \text{LRU}$).
 - `way_of_pos[k]`: An array where the index is the recency position and the value is the way currently at that position.

3: Function Behaviors

Now, to implement LRU IPV we will modify the four main functions used by gem5 in for every policy it implements.

1. **reset()** – Insertion logic

This function will be invoked on the victim way (chosen by getVictim()) to insert a new block.

- a. The function finds the target insertion position, p , from the vector: $p=V[k]$.
- b. To make room at position p , all blocks that are at or less recent than p (i.e., those currently at positions p through $k-2$) must be shifted down one level (towards $k-1$).
- c. The new block is then placed at position p .
- d. The `pos_of_way` and `way_of_pos` mappings are updated for all moved blocks and the new block. Blocks at positions 0 to $p-1$ are not affected.

2. **touch()** – Promotion on a cache hit

On every hit, this function promotes the accessed block.

- a. The block's current recency position, i , is retrieved (e.g., $i = \text{pos_of_way}[\text{hit_way}]$).
- b. Its new target position, p , is determined from the vector: $p=V[i]$.
- c. If $p < i$ (promotion): The block is moved up to position p . The blocks that were between p and $i-1$ are all shifted down by one position to make room.
- d. If $p > i$ (demotion): The block is moved down to position p . The blocks that were between $i+1$ and p are all shifted up by one position to fill the gap.
- e. The `pos_of_way` and `way_of_pos` mappings are updated for all affected blocks.

3. **invalidate()** – Handling invalidations

When a block becomes invalid, its position is removed from the recency order.

- a. Find the position i of the invalidated block.
- b. All blocks below it (at positions $i+1$ through $k-1$) are shifted upward one spot to close the gap.
- c. This ensures the recency positions remain continuous and unique, and the newly-open "invalid" slot naturally moves to the LRU position ($k-1$), making it the next victim.

4. **getVictim()** – Victim selection

When a replacement is needed, this function selects a way to evict.

- a. Following the `invalidate()` logic, any invalid blocks will have been shifted to the end of the recency stack.
- b. Therefore, the function always chooses the block at the true LRU position: $k-1$.
- c. The way to be returned is simply `way_of_pos[k-1]`.

4: Initialization and Configuration

During initialization, the recency mappings are filled in ascending order, that is from position 0 $\rightarrow$ way 0, ... to position $k-1 \rightarrow$ way $k-1$. The vector V can initially follow standard LRU behavior. For validation, this is $V_{LRU}=[0,0,...,0]$ (a $k+1$ length vector of all zeros), which directs both hits and new insertions to the MRU position (position 0). This vector can then be replaced by the optimized IPV vector provided in the paper. The policy object for each block holds a shared pointer to the per-set metadata so that all blocks in a set update the same recency information when accessed, similar to the shared tree data used in tree-PLRU.

So, to summarise the implementation flow as below:

1. Initialize per set metadata that is positions and vector V .
2. On each cache hit, invoke `touch()` → promote block based on V .
3. On each insertion, invoke `reset()` → insert new block at $V[k]$.
4. On invalidation, call `invalidate()` → remove and compact stack.
5. On a miss requiring eviction, call `getVictim()` → evict position $k-1$.